

■ AI RESUMIX PRO — SOURCE CODE DOCUMENT

Complete Python Codebase & Project Specifications

■ README.md

Documentation & LinkedIn Post Template

■ AI Resumix Pro – Intelligent CV & Resume Builder

```
[![Python 3.10+](https://img.shields.io/badge/Python-3.10%2B-3776AB?style=for-the-badge&logo=python&logoColor=white)](https://img.shields.io/badge/Python-3.10%2B-3776AB?style=for-the-badge&logo=python&logoColor=white)
[![Streamlit](https://img.shields.io/badge/Streamlit-FF4B4B?style=for-the-badge&logo=Streamlit&logoColor=white)](https://img.shields.io/badge/Streamlit-FF4B4B?style=for-the-badge&logo=Streamlit&logoColor=white)
[![Google Gemini](https://img.shields.io/badge/Google%20Gemini-4285F4?style=for-the-badge&logo=google&logoColor=white)](https://img.shields.io/badge/Google%20Gemini-4285F4?style=for-the-badge&logo=google&logoColor=white)
[![OpenAI](https://img.shields.io/badge/OpenAI-412991?style=for-the-badge&logo=openai&logoColor=white)](https://img.shields.io/badge/OpenAI-412991?style=for-the-badge&logo=openai&logoColor=white)
[![ReportLab PDF](https://img.shields.io/badge/ReportLab-Vector_PDF-007ACC?style=for-the-badge)](https://img.shields.io/badge/ReportLab-Vector_PDF-007ACC?style=for-the-badge)
[![License: MIT](https://img.shields.io/badge/License-MIT-yellow.svg?style=for-the-badge)](https://img.shields.io/badge/License-MIT-yellow.svg?style=for-the-badge)](LICENSE)
```

AI Resumix Pro is a modern, full-stack Python application that empowers job seekers and engineers to craft high-impact resumes. Featuring a dark glassmorphism Streamlit UI, real-time PDF generation across multiple design themes, automated STAR-formatting, and intelligent keyword matching.

■ Key Features

- **Generative AI Resume Studio**:
 - **AI Executive Summary Generator**: Crafts tailored 60-80 word summary statements matching target roles.
 - **STAR Bullet Point Polisher**: Rewrites raw bullet points into action-driven statements with metrics.
 - **Skill Taxonomy Recommender**: Auto-suggests high-demand technical and soft skills for any job title.
- **ATS Keyword & Score Analyzer**:
 - Paste any job posting description to receive an instant **ATS Match Score (0-100%)**.
 - Highlights matched keywords vs. missing critical industry keywords.
 - 1-Click auto-merge missing keywords into your resume skill list.
- **Vector PDF Export Engine (ReportLab)**:
 - 3 professional visual themes: **Modern Executive**, **Creative Tech**, and **Minimalist Classic**.
 - Crisp typography, balanced margins, printable vector quality.
- **Dual AI Mode + Smart Offline Fallback**:
 - Works with **Google Gemini 1.5** or **OpenAI GPT-3.5/4**.
 - Includes a **Smart Heuristic Engine** out-of-the-box so the app functions seamlessly without an API key!
- **Profile Persistence**: Save and reload resume profiles in JSON format for quick updates.

■ Architecture

```
graph TD
    User[User / Candidate] --> UI[Streamlit Glassmorphism Interface]
    subgraph Core_Engine [Core Engine]
        UI --> Editor[Interactive Resume Form]
        Editor --> AI_Studio[AI Enhancement Studio]
        AI_Studio --> ATS_Analyzer[ATS Keyword Matcher]
    end
    subgraph AI_Processing_Layer [AI Processing Layer]
        AI_Studio --> Router{API Key Configured?}
        Router -- Yes (Gemini/OpenAI) --> CloudAI[LLM APIs]
        Router -- No --> OfflineAI[Smart Heuristic Engine]
    end
```

```

end

subgraph Export Engine
    UI --&gt; PDFGen[■ ReportLab PDF Generator]
    PDFGen --&gt; Themes[■ 3 Modern Visual Themes]
    Themes --&gt; PDF[■ Downloadable Vector PDF]
end
```

■ Quick Start Guide

1. Clone the Repository
```bash
git clone https://github.com/your-username/ai-cv-generator.git
cd ai-cv-generator
```

2. Install Dependencies
```bash
python -m pip install -r requirements.txt
```

3. (Optional) Configure Environment Variables
Copy `.env.example` to `.env` and add your Gemini or OpenAI API keys:
```bash
cp .env.example .env
```

4. Run the Streamlit Application
```bash
python -m streamlit run app.py
```

The web application will launch at `http://localhost:8501`.

■ Ready-to-Post LinkedIn Post Template

> *Copy & paste this exact template to post your project on LinkedIn:*

```markdown
■ Excited to launch my latest Python project: AI Resumix Pro! ■■

Creating a standout resume shouldn't take hours of manual formatting or guessing what ATS scanners want. So I built an

■ Key Features:
■ AI STAR Bullet Rewriter: Transforms weak descriptions into metric-driven accomplishment statements.
■ ATS Keyword & Score Analyzer: Evaluates job description match % and highlights missing key terms.
■ Multi-Theme PDF Engine: Renders crisp vector PDFs across 3 modern design styles (Modern Executive, Creative Tech, Min
■ Dual-Engine AI: Supports Google Gemini, OpenAI, and a zero-config Smart Offline engine.

■■ Built with Python, Streamlit, Google Gemini AI API, and ReportLab.

Check out the code & star the repo on GitHub! ■
[Insert your GitHub Repo Link Here]

#Python #AI #MachineLearning #Streamlit #OpenSource #SoftwareEngineering #CareerGrowth #WebDevelopment #GenerativeAI
```

■ License
Distributed under the MIT License. See `LICENSE` for more information.

```

## ■ requirements.txt

*Project Dependencies Specification*

---

```
streamlit>=1.30.0
reportlab>=4.0.0
google-generativeai>=0.3.0
openai>=1.0.0
python-dotenv>=1.0.0
pillow>=10.0.0
```

## ■ app.py

*Streamlit Main Web Application & Glassmorphism UI*

---

```
import base64
import json
import streamlit as st

from utils import (
 ResumeData, WorkExperience, Education, Project, Certification, get_sample_resume
)
from ai_engine import AIEngine
from pdf_generator import PDFResumeGenerator

Set Streamlit Page Configuration
st.set_page_config(
 page_title="AI Resumix Pro | Professional CV & Resume Builder",
 page_icon="■",
 layout="wide",
 initial_sidebar_state="expanded"
)

Custom Injected CSS for Glassmorphism & High Aesthetic Visuals
CUSTOM_CSS = """
<style>
/* Dark glassmorphism & gradient themes */
.stApp {
 background: linear-gradient(135deg, #0f172a 0%, #1e1b4b 50%, #0f172a 100%);
 color: #f8fafc;
}

/* Header Container */
.header-box {
 background: rgba(30, 41, 59, 0.7);
 backdrop-filter: blur(12px);
 border: 1px solid rgba(255, 255, 255, 0.1);
 border-radius: 16px;
 padding: 24px;
 margin-bottom: 24px;
 text-align: center;
 box-shadow: 0 8px 32px 0 rgba(0, 0, 0, 0.37);
}

.header-title {
 background: linear-gradient(90deg, #38bdf8, #818cf8, #c084fc);
 -webkit-background-clip: text;
 -webkit-text-fill-color: transparent;
 font-size: 2.6rem;
 font-weight: 800;
 margin-bottom: 8px;
}

.header-subtitle {
 color: #94a3b8;
```

```

 font-size: 1.1rem;
 }

 /* Card Container */
 .glass-card {
 background: rgba(30, 41, 59, 0.6);
 backdrop-filter: blur(10px);
 border: 1px solid rgba(255, 255, 255, 0.08);
 border-radius: 12px;
 padding: 20px;
 margin-bottom: 20px;
 }

 /* Custom Metric Badges */
 .metric-badge {
 display: inline-block;
 background: rgba(99, 102, 241, 0.2);
 border: 1px solid rgba(99, 102, 241, 0.4);
 color: #a5b4fc;
 padding: 6px 14px;
 border-radius: 20px;
 font-weight: 600;
 font-size: 0.9rem;
 margin: 4px;
 }

 .missing-badge {
 display: inline-block;
 background: rgba(239, 68, 68, 0.2);
 border: 1px solid rgba(239, 68, 68, 0.4);
 color: #fca5a5;
 padding: 6px 14px;
 border-radius: 20px;
 font-weight: 600;
 font-size: 0.9rem;
 margin: 4px;
 }

 /* Streamlit Buttons Styling */
 .stButton>button {
 border-radius: 8px;
 font-weight: 600;
 transition: all 0.3s ease;
 }
</style>
"""

st.markdown(CUSTOM_CSS, unsafe_allow_html=True)

Initialize Session State
if "resume" not in st.session_state:
 st.session_state.resume = get_sample_resume()

Header Banner
st.markdown("""
<div class="header-box">
 <div class="header-title">■ AI Resumix Pro</div>
 <div class="header-subtitle">Build, Enhance & Tailor Professional CVs Powered by Generative AI</div>
</div>
""", unsafe_allow_html=True)

Sidebar - Settings & Actions
with st.sidebar:
 st.image("https://img.icons8.com/isometric/96/resume.png", width=70)
 st.title("■ AI & Layout Settings")

 provider = st.selectbox(
 "AI Intelligence Mode",
 ["Smart Offline (No Key Needed)", "Google Gemini", "OpenAI"],
 help="Smart Offline mode works immediately without an API key using built-in heuristic algorithms."
)

```

```

)

api_key = ""
if provider in ["Google Gemini", "OpenAI"]:
 api_key = st.text_input(
 f"Enter {provider} API Key",
 type="password",
 help="Your API key stays strictly in memory for this session."
)

st.divider()
st.subheader("■ PDF Visual Theme")
pdf_theme = st.selectbox(
 "Select Resume Theme",
 ["Modern Executive", "Creative Tech", "Minimalist Classic"]
)

st.divider()
st.subheader("■ Quick Actions")
col_s1, col_s2 = st.columns(2)
if col_s1.button("■ Load Sample", use_container_width=True):
 st.session_state.resume = get_sample_resume()
 st.rerun()
if col_s2.button("■ Clear Form", use_container_width=True):
 st.session_state.resume = ResumeData()
 st.rerun()

st.markdown("---")
st.caption("■ Created for LinkedIn Portfolio Showcase")

Initialize AI Engine
ai_engine = AIEngine(provider=provider, api_key=api_key)

Main Navigation Tabs
tab_editor, tab_ai, tab_ats, tab_export = st.tabs([
 "■ Interactive Resume Editor",
 "■ AI Enhancement Studio",
 "■ ATS Keyword & Score Analyzer",
 "■ PDF Download & JSON Sync"
])

TAB 1: INTERACTIVE RESUME EDITOR

with tab_editor:
 res = st.session_state.resume

 st.markdown("### ■ Personal Details")
 c1, c2, c3 = st.columns(3)
 res.full_name = c1.text_input("Full Name", value=res.full_name)
 res.job_title = c2.text_input("Target Job Title", value=res.job_title)
 res.email = c3.text_input("Email Address", value=res.email)

 c4, c5, c6 = st.columns(3)
 res.phone = c4.text_input("Phone Number", value=res.phone)
 res.location = c5.text_input("Location", value=res.location)
 res.linkedin = c6.text_input("LinkedIn Profile URL", value=res.linkedin)

 c7, c8 = st.columns(2)
 res.github = c7.text_input("GitHub Profile URL", value=res.github)
 res.portfolio = c8.text_input("Portfolio Website URL", value=res.portfolio)

 st.markdown("---")
 st.markdown("### ■ Professional Summary")
 res.summary = st.text_area("Executive Summary", value=res.summary, height=110)

 st.markdown("---")
 st.markdown("### ■ Core Competencies & Skills")
 skills_text = st.text_input("Skills (comma-separated)", value=", ".join(res.skills))
 res.skills = [s.strip() for s in skills_text.split(",") if s.strip()]

```

```

st.markdown("---")
st.markdown("### ■ Work Experience")

Add new experience button
if st.button("■ Add Work Experience"):
 res.experiences.append(WorkExperience(company="New Company", role="Role Title"))
 st.rerun()

exp_to_remove = None
for idx, exp in enumerate(res.experiences):
 with st.expander(f"■ {exp.role or 'Position'} at {exp.company or 'Company'}", expanded=(idx == 0)):
 cx1, cx2, cx3 = st.columns(3)
 exp.company = cx1.text_input("Company", value=exp.company, key=f"exp_comp_{idx}")
 exp.role = cx2.text_input("Role Title", value=exp.role, key=f"exp_role_{idx}")
 exp.location = cx3.text_input("Location", value=exp.location, key=f"exp_loc_{idx}")

 cd1, cd2 = st.columns(2)
 exp.start_date = cd1.text_input("Start Date", value=exp.start_date, key=f"exp_start_{idx}")
 exp.end_date = cd2.text_input("End Date", value=exp.end_date, key=f"exp_end_{idx}")

 bullets_str = "\n".join(exp.bullets)
 updated_bullets = st.text_area(
 "Accomplishment Bullets (one per line)",
 value=bullets_str,
 height=100,
 key=f"exp_bullets_{idx}"
)
 exp.bullets = [b.strip() for b in updated_bullets.split("\n") if b.strip()]

 if st.button(f"■ Delete Item", key=f"del_exp_{idx}"):
 exp_to_remove = idx

if exp_to_remove is not None:
 res.experiences.pop(exp_to_remove)
 st.rerun()

st.markdown("---")
st.markdown("### ■ Education & Certifications")

col_edu, col_cert = st.columns(2)
with col_edu:
 st.subheader("Education")
 if st.button("■ Add Education"):
 res.education.append(Education(institution="University Name", degree="Degree"))
 st.rerun()

 edu_to_remove = None
 for idx, edu in enumerate(res.education):
 with st.expander(f"■ {edu.degree} - {edu.institution}", expanded=(idx == 0)):
 edu.institution = st.text_input("Institution", value=edu.institution, key=f"edu_inst_{idx}")
 edu.degree = st.text_input("Degree", value=edu.degree, key=f"edu_deg_{idx}")
 edu.field_of_study = st.text_input("Field of Study", value=edu.field_of_study, key=f"edu_field_{idx}")
 c_e1, c_e2 = st.columns(2)
 edu.start_date = c_e1.text_input("Start Date", value=edu.start_date, key=f"edu_s_{idx}")
 edu.end_date = c_e2.text_input("End Date", value=edu.end_date, key=f"edu_e_{idx}")
 edu.gpa = st.text_input("GPA", value=edu.gpa, key=f"edu_gpa_{idx}")
 if st.button("Delete Education", key=f"del_edu_{idx}"):
 edu_to_remove = idx

 if edu_to_remove is not None:
 res.education.pop(edu_to_remove)
 st.rerun()

with col_cert:
 st.subheader("Certifications")
 if st.button("■ Add Certification"):
 res.certifications.append(Certification(name="Certification Title"))
 st.rerun()

 cert_to_remove = None
 for idx, cert in enumerate(res.certifications):

```

```

 with st.expander(f"■ {cert.name}", expanded=False):
 cert.name = st.text_input("Certification Name", value=cert.name, key=f"cert_n_{idx}")
 cert.issuer = st.text_input("Issuer", value=cert.issuer, key=f"cert_i_{idx}")
 cert.year = st.text_input("Year", value=cert.year, key=f"cert_y_{idx}")
 if st.button("Delete Certification", key=f"del_cert_{idx}"):
 cert_to_remove = idx
 if cert_to_remove is not None:
 res.certifications.pop(cert_to_remove)
 st.rerun()

TAB 2: AI ENHANCEMENT STUDIO

with tab_ai:
 st.markdown("## ■ AI Enhancement Studio")
 st.info(f"Currently active AI Engine: **{provider}**")

 col_ai1, col_ai2 = st.columns(2)

 with col_ai1:
 st.subheader("1. AI Executive Summary Generator")
 st.write("Generate a high-impact, tailored executive summary based on your profile.")
 if st.button("■ Generate AI Summary", use_container_width=True):
 exp_text = "; ".join([f"{e.role} at {e.company}" for e in res.experiences])
 with st.spinner("AI is crafting your summary..."):
 gen_summary = ai_engine.generate_summary(
 name=res.full_name,
 job_title=res.job_title,
 skills=res.skills,
 experience_summary=exp_text
)
 st.session_state["gen_summary_temp"] = gen_summary

 if "gen_summary_temp" in st.session_state:
 st.success("Generated Summary:")
 st.write(st.session_state["gen_summary_temp"])
 if st.button("■ Apply to Resume Summary"):
 res.summary = st.session_state["gen_summary_temp"]
 st.success("Summary updated!")
 st.rerun()

 with col_ai2:
 st.subheader("2. AI Bullet Point Rewriter")
 st.write("Transform weak bullet points into metric-driven STAR accomplishment statements.")

 raw_bullet = st.text_area(
 "Paste bullet point to enhance",
 placeholder="e.g. Responsible for managing the company database and building APIs."
)
 if st.button("■ Polish & Enhance Bullet", use_container_width=True):
 if raw_bullet.strip():
 with st.spinner("Rewriting with active verbs & metrics..."):
 enhanced = ai_engine.enhance_bullet_point(raw_bullet, target_role=res.job_title)
 st.session_state["enhanced_bullet_temp"] = enhanced
 else:
 st.warning("Please enter a bullet point to enhance.")

 if "enhanced_bullet_temp" in st.session_state:
 st.success("High-Impact Result:")
 st.code(st.session_state["enhanced_bullet_temp"], language="text")

 st.markdown("---")
 st.subheader("3. AI Skill Recommender")
 st.write("Get instant skill suggestions tailored to your target job title.")
 if st.button("■ Recommend Industry Skills"):
 with st.spinner("Searching skill taxonomies..."):
 suggested = ai_engine.suggest_skills(res.job_title)
 st.session_state["suggested_skills"] = suggested

 if "suggested_skills" in st.session_state:

```

```

st.write("***Recommended Skills:***")
badges_html = "".join([f'{s}' for s in st.session_state["suggested_skills"]])
st.markdown(badges_html, unsafe_allow_html=True)
if st.button("■ Merge Suggested Skills into Resume"):
 new_skills = list(set(res.skills + st.session_state["suggested_skills"]))
 res.skills = new_skills
 st.success("Skills merged successfully!")
 st.rerun()

TAB 3: ATS KEYWORD & SCORE ANALYZER

with tab_ats:
 st.markdown("## ■ ATS Keyword & Match Analyzer")
 st.write("Paste a target job posting description to evaluate ATS match percentage and identify missing keywords.")

 job_desc = st.text_area("Paste Target Job Description (JD)", height=160, placeholder="Paste the job advertisement text here")

 if st.button("■ Run ATS Match Analysis", use_container_width=True):
 # Build composite resume text
 exp_text = " ".join([" ".join(e.bullets) for e in res.experiences])
 full_res_text = f"{res.job_title} {res.summary} {' '.join(res.skills)} {exp_text}"

 with st.spinner("Analyzing keyword frequency & semantic density..."):
 ats_res = ai_engine.analyze_ats(full_res_text, job_desc)
 st.session_state["ats_result"] = ats_res

 if "ats_result" in st.session_state:
 ats = st.session_state["ats_result"]

 col_m1, col_m2 = st.columns([1, 2])
 with col_m1:
 st.metric("ATS Compatibility Score", f"{ats['score']}%")
 st.progress(ats['score'] / 100.0)
 with col_m2:
 st.markdown(f"***Analysis Summary*: {ats['summary']}")

 st.markdown("### ■ Matched Keywords")
 if ats["matched_keywords"]:
 matched_html = "".join([f'{w}' for w in ats["matched_keywords"]])
 st.markdown(matched_html, unsafe_allow_html=True)
 else:
 st.write("No exact key terms matched yet.")

 st.markdown("### ■ Missing Keywords (Recommended to Add)")
 if ats["missing_keywords"]:
 missing_html = "".join([f'{w}' for w in ats["missing_keywords"]])
 st.markdown(missing_html, unsafe_allow_html=True)
 if st.button("■ Auto-Add Missing Keywords to Skills"):
 res.skills = list(set(res.skills + ats["missing_keywords"]))
 st.success("Added missing keywords to your skill list!")
 st.rerun()

 st.markdown("### ■ Optimization Tips")
 for tip in ats["tips"]:
 st.write(f"- {tip}")

TAB 4: PDF DOWNLOAD & JSON SYNC

with tab_export:
 st.markdown("## ■ Export & Sync Studio")

 col_p1, col_p2 = st.columns([1, 1])

 pdf_gen = PDFResumeGenerator(theme_name=pdf_theme)
 pdf_bytes = pdf_gen.generate(st.session_state.resume)

 with col_p1:
 st.subheader("■ Download Vector PDF")

```



```

st.write(f"Theme Selected: **{pdf_theme}**")

file_name = f"Resume_{st.session_state.resume.full_name.replace(' ', '_')}.pdf"
st.download_button(
 label="■ Download Professional Resume PDF",
 data=pdf_bytes,
 file_name=file_name,
 mime="application/pdf",
 use_container_width=True
)

st.markdown("---")
st.subheader("■ Backup & Sync Resume Data")

Save JSON
json_str = json.dumps(st.session_state.resume.to_dict(), indent=2)
st.download_button(
 label="■ Export Resume Profile (JSON)",
 data=json_str,
 file_name="resume_profile.json",
 mime="application/json",
 use_container_width=True
)

Load JSON
uploaded_file = st.file_uploader("■ Import Saved Resume Profile (JSON)", type=["json"])
if uploaded_file is not None:
 try:
 data_dict = json.load(uploaded_file)
 st.session_state.resume = ResumeData.from_dict(data_dict)
 st.success("Resume data imported successfully!")
 st.rerun()
 except Exception as e:
 st.error(f"Error loading JSON: {e}")

with col_p2:
 st.subheader("■ Live PDF Preview")
 base64_pdf = base64.b64encode(pdf_bytes).decode('utf-8')
 pdf_display = f'<iframe src="data:application/pdf;base64,{base64_pdf}" width="100%" height="550" type="appli'
 st.markdown(pdf_display, unsafe_allow_html=True)

```

## ■ ai\_engine.py

*AI Engine Wrapper (Gemini, OpenAI & Smart Fallback)*

```

import os
import re
import random
from typing import Dict, List, Any, Tuple

Try importing AI SDKs gracefully
try:
 import google.generativeai as genai
 HAS_GEMINI = True
except ImportError:
 HAS_GEMINI = False

try:
 import openai
 HAS_OPENAI = True
except ImportError:
 HAS_OPENAI = False

class AIEngine:
 def __init__(self, provider: str = "Smart Offline", api_key: str = ""):
 self.provider = provider

```

```

self.api_key = api_key.strip()

if self.provider == "Google Gemini" and self.api_key and HAS_GEMINI:
 genai.configure(api_key=self.api_key)
 self.model = genai.GenerativeModel('gemini-1.5-flash')
elif self.provider == "OpenAI" and self.api_key and HAS_OPENAI:
 self.client = openai.OpenAI(api_key=self.api_key)

def generate_summary(self, name: str, job_title: str, skills: List[str], experience_summary: str) -> str:
 """Generates a professional executive summary."""
 prompt = f"""
 Write a high-impact, professional executive summary (3-4 sentences, ~60-80 words) for a CV/Resume.
 Candidate Name: {name}
 Target Job Title: {job_title}
 Key Skills: {' '.join(skills[:8])}
 Brief Experience background: {experience_summary}

 Requirements:
 - Use strong action verbs and industry-standard keywords.
 - Highlight value delivered, leadership/collaboration, and technical mastery.
 - Do NOT use filler words or generic fluff.
 - Write in third-person professional tone or first-person implied (no "I am a...").
 """

 if self.provider == "Google Gemini" and self.api_key and HAS_GEMINI:
 try:
 response = self.model.generate_content(prompt)
 return response.text.strip()
 except Exception as e:
 pass # fallback to smart engine

 elif self.provider == "OpenAI" and self.api_key and HAS_OPENAI:
 try:
 res = self.client.chat.completions.create(
 model="gpt-3.5-turbo",
 messages=[{"role": "user", "content": prompt}],
 max_tokens=200,
 temperature=0.7
)
 return res.choices[0].message.content.strip()
 except Exception as e:
 pass # fallback to smart engine

 # Smart Fallback Engine
 top_skills = ", ".join(skills[:4]) if skills else "modern technical stacks"
 return (
 f"Results-oriented {job_title or 'Professional'} with proven expertise in {top_skills}. "
 f"Demonstrated history of driving technical innovation, optimizing system performance, and scaling operations. "
 f"Adept at collaborating across cross-functional teams to deliver scalable, high-business-impact solutions."
)

def enhance_bullet_point(self, raw_bullet: str, target_role: str = "") -> str:
 """Transforms a weak or simple bullet point into a high-impact action statement with metrics."""
 if not raw_bullet.strip():
 return raw_bullet

 prompt = f"""
 Rewrite the following resume bullet point into a high-impact, STAR-formatted (Situation, Task, Action, Result)
 Target Job Role: {target_role}
 Original Bullet Point: "{raw_bullet}"

 Rules:
 - Start with a strong active power verb (e.g. Architected, Spearheaded, Optimized, Orchestrated).
 - Include realistic quantifiable metrics or impact (e.g. %, speed improvement, revenue, time saved).
 - Keep it concise (15-25 words).
 - Output ONLY the rewritten bullet point without quotation marks or bullet symbols.
 """

 if self.provider == "Google Gemini" and self.api_key and HAS_GEMINI:
 try:

```

```

 response = self.model.generate_content(prompt)
 return response.text.strip().lstrip("•- ").strip()
 except Exception:
 pass

elif self.provider == "OpenAI" and self.api_key and HAS_OPENAI:
 try:
 res = self.client.chat.completions.create(
 model="gpt-3.5-turbo",
 messages=[{"role": "user", "content": prompt}],
 max_tokens=100,
 temperature=0.7
)
 return res.choices[0].message.content.strip().lstrip("•- ").strip()
 except Exception:
 pass

Smart Heuristic Rewriter (Offline)
power_verbs = ["Spearheaded", "Architected", "Optimized", "Streamlined", "Orchestrated", "Engineered"]
metrics = ["by 35%", "reducing processing time by 40%", "saving 15+ engineering hours weekly", "boosting platform"]

verb = random.choice(power_verbs)
metric = random.choice(metrics)
clean_raw = raw_bullet.strip().rstrip(".")

Remove weak leading words if any
clean_raw = re.sub(r'^((worked on|helped with|responsible for|did|made|built|created)\s+)', '', clean_raw, flags=re.IGNORECASE)

return f"{verb} {clean_raw[0].lower() + clean_raw[1:] if clean_raw else 'key initiatives'}, {metric}."

def analyze_atc(self, resume_text: str, job_description: str) -> Dict[str, Any]:
 """Analyzes resume text against a target job description for ATS compatibility."""
 if not job_description.strip():
 return {
 "score": 85,
 "summary": "Please provide a target Job Description to run detailed ATS comparison.",
 "matched_keywords": ["Python", "Engineering", "Architecture", "API", "Optimization"],
 "missing_keywords": ["CI/CD", "Cloud Architecture", "Agile Methodologies"],
 "tips": [
 "Include quantifiable metrics in every bullet point.",
 "Tailor your skill section keywords to match job posting titles directly."
]
 }

Extract words for heuristic matching
def extract_keywords(text: str) -> set:
 words = re.findall(r'\b[A-Za-z0-9+#+.-]{3,}\b', text)
 stopwords = {"and", "the", "for", "with", "that", "this", "from", "have", "you", "your", "are", "will", "ou"}
 return {w.strip().lower() for w in words if w.lower() not in stopwords and len(w) > 2}

jd_words = extract_keywords(job_description)
resume_words = extract_keywords(resume_text)

matched = jd_words.intersection(resume_words)
missing = jd_words.difference(resume_words)

match_ratio = len(matched) / (len(jd_words) if jd_words else 1)
calculated_score = min(98, max(45, int(match_ratio * 100 + 20)))

Convert back to readable sample words
display_matched = sorted([w.capitalize() for w in list(matched)[:10]])
display_missing = sorted([w.capitalize() for w in list(missing)[:8]])

return {
 "score": calculated_score,
 "summary": f"Your resume has a **{calculated_score}% ATS keyword alignment** with the targeted job description.",
 "matched_keywords": display_matched,
 "missing_keywords": display_missing,
 "tips": [
 f"Add these missing key terms into your skills or bullet points: {'', '.join(display_missing[:4])}'.",
]
}

```

```

 "Ensure your job titles match standard industry naming conventions.",
 "Use bulleted list formatting over dense multi-line paragraphs for better ATS parser readability."
]
}

def suggest_skills(self, job_title: str) -> List[str]:
 """Suggests relevant technical and soft skills based on job title."""
 title_lower = job_title.lower()
 if "data" in title_lower or "ai" in title_lower or "machine learning" in title_lower:
 return ["Python", "PyTorch", "TensorFlow", "Pandas", "Scikit-Learn", "SQL", "LLMs", "LangChain", "Vector DB"]
 elif "frontend" in title_lower or "web" in title_lower or "react" in title_lower:
 return ["React.js", "Next.js", "TypeScript", "JavaScript", "HTML5/CSS3", "Tailwind CSS", "Redux", "REST API"]
 elif "backend" in title_lower or "cloud" in title_lower or "devops" in title_lower:
 return ["Python", "Go", "Java", "Docker", "Kubernetes", "AWS", "FastAPI", "PostgreSQL", "Redis", "Kafka", ""]
 else:
 return ["Python", "Strategic Planning", "Project Management", "Agile/Scrum", "Data Analysis", "System Design"]

```

## ■ pdf\_generator.py

*ReportLab Vector PDF Resume Generator Engine*

```

import io
from typing import List, Dict, Any
from reportlab.lib.pagesizes import letter
from reportlab.lib import colors
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.platypus import (
 SimpleDocTemplate, Paragraph, Spacer, HRFFlowable, Table, TableStyle, KeepTogether
)

from utils import ResumeData

class PDFResumeGenerator:
 """Generates styled vector PDFs from ResumeData using ReportLab."""

 THEME_COLORS = {
 "Modern Executive": {
 "primary": colors.HexColor("#1e3a8a"), # Deep Navy Blue
 "secondary": colors.HexColor("#3b82f6"), # Bright Blue
 "text": colors.HexColor("#1f2937"), # Charcoal Gray
 "accent": colors.HexColor("#f3f4f6"), # Light Gray Background
 "divider": colors.HexColor("#cbd5e1")
 },
 "Creative Tech": {
 "primary": colors.HexColor("#0f766e"), # Dark Teal
 "secondary": colors.HexColor("#0d9488"), # Teal
 "text": colors.HexColor("#18181b"), # Dark Zinc
 "accent": colors.HexColor("#ccfbf1"), # Mint Tint
 "divider": colors.HexColor("#99f6e4")
 },
 "Minimalist Classic": {
 "primary": colors.HexColor("#18181b"), # Almost Black
 "secondary": colors.HexColor("#52525b"), # Medium Zinc
 "text": colors.HexColor("#27272a"), # Soft Charcoal
 "accent": colors.HexColor("#f4f4f5"), # Off-White
 "divider": colors.HexColor("#e4e4e7")
 }
 }

 def __init__(self, theme_name: str = "Modern Executive"):
 self.theme_name = theme_name if theme_name in self.THEME_COLORS else "Modern Executive"
 self.colors_cfg = self.THEME_COLORS[self.theme_name]

 def _get_styles(self):
 styles = getSampleStyleSheet()

```

```

title_style = ParagraphStyle(
 'CVName',
 parent=styles['Normal'],
 fontName='Helvetica-Bold',
 fontSize=22,
 leading=26,
 textColor=self.colors_cfg["primary"],
 spaceAfter=2
)

job_title_style = ParagraphStyle(
 'CVJobTitle',
 parent=styles['Normal'],
 fontName='Helvetica-Bold',
 fontSize=12,
 leading=15,
 textColor=self.colors_cfg["secondary"],
 spaceAfter=6
)

contact_style = ParagraphStyle(
 'CVContact',
 parent=styles['Normal'],
 fontName='Helvetica',
 fontSize=9,
 leading=12,
 textColor=self.colors_cfg["text"],
 alignment=0 # Left aligned
)

section_heading = ParagraphStyle(
 'CVSectionHeading',
 parent=styles['Normal'],
 fontName='Helvetica-Bold',
 fontSize=13,
 leading=16,
 textColor=self.colors_cfg["primary"],
 spaceBefore=10,
 spaceAfter=4
)

body_style = ParagraphStyle(
 'CVBody',
 parent=styles['Normal'],
 fontName='Helvetica',
 fontSize=9.5,
 leading=13.5,
 textColor=self.colors_cfg["text"],
 spaceAfter=4
)

bullet_style = ParagraphStyle(
 'CVBullet',
 parent=styles['Normal'],
 fontName='Helvetica',
 fontSize=9.5,
 leading=13.5,
 textColor=self.colors_cfg["text"],
 leftIndent=12,
 spaceAfter=3
)

bold_item_title = ParagraphStyle(
 'CVItemTitle',
 parent=styles['Normal'],
 fontName='Helvetica-Bold',
 fontSize=10.5,
 leading=13,
 textColor=self.colors_cfg["primary"]
)

```

```

sub_item_title = ParagraphStyle(
 'CVSubItemTitle',
 parent=styles['Normal'],
 fontName='Helvetica-Oblique',
 fontSize=9.5,
 leading=12,
 textColor=self.colors_cfg["secondary"]
)

right_date_style = ParagraphStyle(
 'CVRightDate',
 parent=styles['Normal'],
 fontName='Helvetica-Bold',
 fontSize=9.5,
 leading=12,
 textColor=self.colors_cfg["secondary"],
 alignment=2 # Right aligned
)

return {
 "title": title_style,
 "job_title": job_title_style,
 "contact": contact_style,
 "heading": section_heading,
 "body": body_style,
 "bullet": bullet_style,
 "item_title": bold_item_title,
 "sub_title": sub_item_title,
 "date": right_date_style
}

def generate(self, data: ResumeData) -> bytes:
 buffer = io.BytesIO()
 doc = SimpleDocTemplate(
 buffer,
 pageSize=letter,
 leftMargin=36,
 rightMargin=36,
 topMargin=36,
 bottomMargin=36
)

 st = self._get_styles()
 story = []

 # --- HEADER SECTION ---
 story.append(Paragraph(data.full_name, st["title"]))
 if data.job_title:
 story.append(Paragraph(data.job_title.upper(), st["job_title"]))

 # Contact line
 contact_parts = []
 if data.email: contact_parts.append(data.email)
 if data.phone: contact_parts.append(data.phone)
 if data.location: contact_parts.append(data.location)
 if data.linkedin: contact_parts.append(f"LinkedIn: {data.linkedin}")
 if data.github: contact_parts.append(f"GitHub: {data.github}")
 if data.portfolio: contact_parts.append(f"Portfolio: {data.portfolio}")

 contact_text = " • ".join(contact_parts)
 story.append(Paragraph(contact_text, st["contact"]))
 story.append(Spacer(1, 6))

 # Colored divider bar
 story.append(HRFlowable(
 width="100%",
 thickness=2,
 color=self.colors_cfg["primary"],
 spaceAfter=8,
 spaceBefore=2
))

```

```

))

--- EXECUTIVE SUMMARY ---
if data.summary:
 story.append(Paragraph("PROFESSIONAL SUMMARY", st["heading"]))
 story.append(HRFlowable(width="100%", thickness=0.5, color=self.colors_cfg["divider"], spaceAfter=4, spaceBefore=4))
 story.append(Paragraph(data.summary, st["body"]))
 story.append(Spacer(1, 6))

--- SKILLS SECTION ---
if data.skills:
 story.append(Paragraph("CORE COMPETENCIES & TECHNICAL SKILLS", st["heading"]))
 story.append(HRFlowable(width="100%", thickness=0.5, color=self.colors_cfg["divider"], spaceAfter=4, spaceBefore=4))
 skills_formatted = " • ".join(data.skills)
 story.append(Paragraph(f"Key Skills: {skills_formatted}", st["body"]))
 story.append(Spacer(1, 6))

--- WORK EXPERIENCE ---
if data.experiences:
 story.append(Paragraph("WORK EXPERIENCE", st["heading"]))
 story.append(HRFlowable(width="100%", thickness=0.5, color=self.colors_cfg["divider"], spaceAfter=4, spaceBefore=4))

 for exp in data.experiences:
 date_str = f"{exp.start_date} – {exp.end_date}" if exp.end_date else exp.start_date
 loc_str = f"({exp.location})" if exp.location else ""

 header_table_data = [
 [
 Paragraph(f"{exp.role}", st["item_title"]),
 Paragraph(date_str, st["date"])
],
 [
 Paragraph(f"{exp.company}{loc_str}", st["sub_title"]),
 Paragraph("", st["date"])
]
]

 t = Table(header_table_data, colWidths=[380, 160])
 t.setStyle(TableStyle([
 ('VALIGN', (0, 0), (-1, -1), 'TOP'),
 ('LEFTPADDING', (0, 0), (-1, -1), 0),
 ('RIGHTPADDING', (0, 0), (-1, -1), 0),
 ('BOTTOMPADDING', (0, 0), (-1, -1), 1),
 ('TOPPADDING', (0, 0), (-1, -1), 1),
]))

 story.append(t)
 story.append(Spacer(1, 3))

 for b in exp.bullets:
 if b.strip():
 story.append(Paragraph(f"• {b.strip()}", st["bullet"]))
 story.append(Spacer(1, 6))

--- PROJECTS SECTION ---
if data.projects:
 story.append(Paragraph("FEATURED PROJECTS", st["heading"]))
 story.append(HRFlowable(width="100%", thickness=0.5, color=self.colors_cfg["divider"], spaceAfter=4, spaceBefore=4))

 for proj in data.projects:
 tech_info = f" [{proj.tech_stack}]" if proj.tech_stack else ""
 link_info = f" ({proj.link})" if proj.link else ""

 header_table_data = [
 [
 Paragraph(f"{proj.title}{tech_info}", st["item_title"]),
 Paragraph(link_info, st["date"])
]
]

 t = Table(header_table_data, colWidths=[380, 160])

```

```

t.setStyle(TableStyle([
 ('VALIGN', (0, 0), (-1, -1), 'TOP'),
 ('LEFTPADDING', (0, 0), (-1, -1), 0),
 ('RIGHTPADDING', (0, 0), (-1, -1), 0),
 ('BOTTOMPADDING', (0, 0), (-1, -1), 1),
 ('TOPPADDING', (0, 0), (-1, -1), 1),
]))
story.append(t)
if proj.description:
 story.append(Paragraph(proj.description, st["body"]))
for b in proj.bullets:
 if b.strip():
 story.append(Paragraph(f"• {b.strip()}", st["bullet"]))
story.append(Spacer(1, 5))

--- EDUCATION SECTION ---
if data.education:
 story.append(Paragraph("EDUCATION", st["heading"]))
 story.append(HRFlowable(width="100%", thickness=0.5, color=self.colors_cfg["divider"], spaceAfter=4, spaceBefore=4))

 for edu in data.education:
 date_str = f"{edu.start_date} - {edu.end_date}" if edu.end_date else edu.start_date
 gpa_str = f" (GPA: {edu.gpa})" if edu.gpa else ""
 degree_full = f"{edu.degree} in {edu.field_of_study}" if edu.field_of_study else edu.degree

 header_table_data = [
 [
 Paragraph(f"{edu.institution}", st["item_title"]),
 Paragraph(date_str, st["date"])
],
 [
 Paragraph(f"{degree_full}{gpa_str}", st["sub_title"]),
 Paragraph("", st["date"])
]
]
 t = Table(header_table_data, colWidths=[380, 160])
 t.setStyle(TableStyle([
 ('VALIGN', (0, 0), (-1, -1), 'TOP'),
 ('LEFTPADDING', (0, 0), (-1, -1), 0),
 ('RIGHTPADDING', (0, 0), (-1, -1), 0),
 ('BOTTOMPADDING', (0, 0), (-1, -1), 1),
 ('TOPPADDING', (0, 0), (-1, -1), 1),
]))
 story.append(t)
 story.append(Spacer(1, 4))

--- CERTIFICATIONS ---
if data.certifications:
 story.append(Paragraph("CERTIFICATIONS & AWARDS", st["heading"]))
 story.append(HRFlowable(width="100%", thickness=0.5, color=self.colors_cfg["divider"], spaceAfter=4, spaceBefore=4))

 cert_lines = []
 for cert in data.certifications:
 yr = f" ({cert.year})" if cert.year else ""
 issuer = f" - {cert.issuer}" if cert.issuer else ""
 cert_lines.append(f"{cert.name}{yr}{issuer}")

 story.append(Paragraph(" • ".join(cert_lines), st["body"]))

doc.build(story)
pdf_bytes = buffer.getvalue()
buffer.close()
return pdf_bytes

```

## ■ utils.py

*Data Models, Sample Data & Helper Utilities*



---

```

import json
from dataclasses import dataclass, field, asdict
from typing import List, Dict, Any

@dataclass
class WorkExperience:
 company: str = ""
 role: str = ""
 location: str = ""
 start_date: str = ""
 end_date: str = ""
 bullets: List[str] = field(default_factory=list)

@dataclass
class Education:
 institution: str = ""
 degree: str = ""
 field_of_study: str = ""
 start_date: str = ""
 end_date: str = ""
 gpa: str = ""

@dataclass
class Project:
 title: str = ""
 description: str = ""
 tech_stack: str = ""
 link: str = ""
 bullets: List[str] = field(default_factory=list)

@dataclass
class Certification:
 name: str = ""
 issuer: str = ""
 year: str = ""

@dataclass
class ResumeData:
 full_name: str = ""
 job_title: str = ""
 email: str = ""
 phone: str = ""
 location: str = ""
 linkedin: str = ""
 github: str = ""
 portfolio: str = ""
 summary: str = ""
 skills: List[str] = field(default_factory=list)
 experiences: List[WorkExperience] = field(default_factory=list)
 education: List[Education] = field(default_factory=list)
 projects: List[Project] = field(default_factory=list)
 certifications: List[Certification] = field(default_factory=list)

 def to_dict(self) -> Dict[str, Any]:
 return asdict(self)

 @classmethod
 def from_dict(cls, data: Dict[str, Any]) -> 'ResumeData':
 exp_list = [WorkExperience(**exp) for exp in data.get("experiences", [])]
 edu_list = [Education(**edu) for edu in data.get("education", [])]
 proj_list = [Project(**proj) for proj in data.get("projects", [])]
 cert_list = [Certification(**cert) for cert in data.get("certifications", [])]

 return cls(
 full_name=data.get("full_name", ""),
 job_title=data.get("job_title", ""),
 email=data.get("email", ""),
 phone=data.get("phone", ""),
 location=data.get("location", ""),

```

```

 linkedin=data.get("linkedin", ""),
 github=data.get("github", ""),
 portfolio=data.get("portfolio", ""),
 summary=data.get("summary", ""),
 skills=data.get("skills", []),
 experiences=exp_list,
 education=edu_list,
 projects=proj_list,
 certifications=cert_list
)

def get_sample_resume() -> ResumeData:
 """Returns a rich, professional sample resume profile for quick testing and demos."""
 return ResumeData(
 full_name="Alex R. Vance",
 job_title="Senior AI & Full Stack Software Engineer",
 email="alex.vance@techstudio.io",
 phone="+1 (555) 382-9102",
 location="San Francisco, CA (Open to Remote)",
 linkedin="linkedin.com/in/alexvance-ai",
 github="github.com/alexvance-dev",
 portfolio="alexvance.dev",
 summary="Results-driven Senior AI & Full-Stack Engineer with 6+ years of experience architecting high-throughput systems and leading cross-functional teams.",
 skills=[
 "Python", "TypeScript", "PyTorch", "OpenAI API", "LangChain", "FastAPI",
 "React.js", "Next.js", "Docker", "Kubernetes", "AWS (EC2, Lambda, S3)",
 "PostgreSQL", "Redis", "Vector DBs (Pinecone, Qdrant)", "CI/CD", "Git"
],
 experiences=[
 WorkExperience(
 company="Nexus AI Innovations",
 role="Senior AI Solutions Engineer",
 location="San Francisco, CA",
 start_date="Jan 2023",
 end_date="Present",
 bullets=[
 "Engineered enterprise LLM orchestration pipeline using Python, LangChain, and Qdrant vector database.",
 "Architected scalable microservices handling 2M+ daily requests on AWS EKS with 99.99% uptime.",
 "Mentored a team of 5 junior/mid-level engineers in clean code practices, unit testing, and prompt engineering."
]
),
 WorkExperience(
 company="Apex Cloud Systems",
 role="Full Stack Software Engineer",
 location="Austin, TX",
 start_date="Jun 2020",
 end_date="Dec 2022",
 bullets=[
 "Developed high-converting React/TypeScript analytics dashboard, driving a 30% increase in enterprise lead generation.",
 "Designed RESTful & GraphQL APIs in FastAPI and PostgreSQL, optimizing query speeds by 3.5x using Redis caching.",
 "Implemented automated CI/CD deployment pipelines via GitHub Actions, reducing release cycle time from 2 weeks to 2 days."
]
)
],
 education=[
 Education(
 institution="University of California, Berkeley",
 degree="Bachelor of Science",
 field_of_study="Computer Science & Data Science",
 start_date="Sep 2016",
 end_date="May 2020",
 gpa="3.88 / 4.0"
)
],
 projects=[
 Project(
 title="AI Knowledge Graph Synthesizer",
 description="An open-source tool that converts raw unstructured documents into interactive semantic knowledge graphs.",
 tech_stack="Python, PyTorch, Streamlit, NetworkX, Neo4j",
 link="github.com/alexvance-dev/graph-synthesizer",
)
]
)

```

```

 bullets=[
 "Starred 1.2k+ times on GitHub and featured in PyTorch Weekly.",
 "Implemented semantic chunking algorithms that improved graph entity extraction accuracy by 28%."
]
),
],
certifications=[
 Certification(
 name="AWS Certified Solutions Architect – Associate",
 issuer="Amazon Web Services",
 year="2023"
),
 Certification(
 name="Deep Learning Specialization",
 issuer="DeepLearning.AI (Coursera)",
 year="2022"
)
]
)

```

## ■ .env.example

*Environment Variables Template*

---

```

AI Provider API Keys (Optional - App includes built-in Smart Offline Engine)
GEMINI_API_KEY=your_google_gemini_api_key_here
OPENAI_API_KEY=your_openai_api_key_here

```